

Herald



Herald — commons_constitution Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for <code>commons_constitution</code> — Herald's L1 governance/compliance engine. Documents the real source API: the <code>Evaluator</code> extension point + concurrent-safe <code>Registry</code>, the <code>BundleHash</code> captureer (stable transition detection), the <code>Modeladder</code> allow/warn/enforce ladder + its <code>Memory/Postgres</code> backends, the <code>ConstitutionStore</code> transition gate + <code>AuditStore</code> write-through with their <code>Memory/Postgres</code> backends, the <code>MemoryBus</code> in-process <code>EventBus</code>, the 13 typed emit helpers on <code>EventEmitter</code> (12 governance classes per spec §42.2 + 1 operational <code>queue.dead_letter</code> per HRD-090), the <code>Runner</code> orchestrator, and the <code>CloudEvents v1.0 ToCloudEvent/FromCloudEvent</code> adapter. Shows how cherald binds it for <code>GET /v1/compliance</code>, the <code>/v1/compliance/modes</code> admin REST surface, and the <code>POST /v1/compliance/evaluate</code> write side. ANTI-BLUFF: every type, interface, and signature in this guide is read directly from <code>commons_constitution/*.go</code> as of this revision — nothing is inferred from the spec alone.
Issues	(none specific to this guide)
Continuation	bump when the Redis read-cache <code>Modeladder</code> wrapper (M3, referenced in <code>ladder.go</code> 's doc comment as <code>ladder/redis_cache.go</code>) lands, and when a real <code>Subjects</code> enumerator / periodic sweep (HRD-025 scherald) replaces the caller-driven <code>EvaluateSubject</code> path.

Table of contents

- [§1. Overview — how Herald binds to the constitution governance plane](#)
- [§2. The Evaluator + Registry framework](#)
- [§3. BundleHash and the captureer](#)
- [§4. The ConstitutionStore transition gate](#)

- [§5. The Modeladder \(allow / warn / enforce\)](#)
 - [§6. Event classes and the typed emit helpers](#)
 - [§7. The EventBus shim \(MemoryBus\)](#)
 - [§8. The AuditStore write-through](#)
 - [§9. The Runner orchestrator](#)
 - [§10. The CloudEvents v1.0 adapter](#)
 - [§11. How cherald consumes the module](#)
 - [§12. Backends summary \(Memory vs Postgres\)](#)
 - [§13. Testing notes](#)
 - [§14. References](#)
-

§1. Overview — how Herald binds to the constitution governance plane

`commons_constitution` (Go package `constitution`, module path `github.com/vasic-digital/herald/commons_constitution`) is Herald's **L1 governance/compliance engine**. It is the substrate every Herald constitution rule (e.g. §11.4.10 credential-leak, §11.4.73 spec-revision drift) evaluates through, and the path by which a constitution-rule decision becomes a fan-out-able governance event.

Per the package doc (`doc.go`) and `go.mod`, the package exposes:

1. an `Evaluator + Registry` framework — the primary extension point; one `Evaluator` per constitution rule;
2. a `BundleHash captureer` — the SHA-256 of the rendered `Constitution.md` bundle, carried on every event for replayability;
3. a `ConstitutionStore` transition gate — the persisted (`tenant, rule, subject`) state and the three-axis transition computation;
4. a `Modeladder` — the per-tenant per-rule `allow/warn/enforce` enforcement-mode lookup;
5. an `AuditStore` — the append-only audit trail written on every emitted/warned transition;
6. the **13 typed emit helpers** on `EventEmitter` — 12 governance classes (spec §42.2) plus the operational `queue.dead_letter`;
7. a `MemoryBus` in-process `EventBus` shim whose interface mirrors the Helix `digital.vasic.eventbus` shape;
8. a `Runner` orchestrator composing all of the above into the canonical `evaluate→record→gate→emit→audit` flow;
9. a **CloudEvents v1.0 adapter** (`ToCloudEvent / FromCloudEvent`) for HTTP egress.

The event namespace is fixed: `EventNamespace = "digital.vasic.herald.constitution"`, and every emitted `ce-type` is `EventNamespace + "." + <class>`.

The only non-stdlib dependencies (`commons_constitution/go.mod`) are `github.com/cloudevents/sdk-go/v2 v2.15.2`, `github.com/google/uuid v1.6.0`, and `digital.vasic.database` (the Helix-stack DB module, wired via `replace digital.vasic.database => ../submodules/database` for the Postgres backends).

The shipped backends are split **Memory (M1, test-only)** vs **Postgres (M2/M3, RLS-guarded production)** for both `Modeladder`, `ConstitutionStore`, and `AuditStore` — see §12.

§2. The Evaluator + Registry framework

§2.1 The four enums

`evaluator.go` defines four small enums that the whole package keys on:

```
type Severity
    int    // SeverityLow, SeverityMiddle, SeverityHigh,
           SeverityCritical
type Decision int    // DecisionPass, DecisionWarn, DecisionFail,
                     DecisionError, DecisionSkip
type Mode int        // (in ladder.go) ModeAllow, ModeWarn,
                     ModeEnforce
```

- `Severity.String()` → low/middle/high/critical). `SeverityCritical` is the push-trigger tier; the rest are sweep-trigger.
- `Decision.String()` → pass/warn/fail/error/skip). `DecisionError` means the evaluator itself failed (panic/network/config) — it is **not** a rule violation. `DecisionSkip` means the evaluator declined (e.g. bundle unreadable).

§2.2 Subject and Result

```
type Subject struct {
    Kind string // "file" / "repo" / "commit" / "tenant" /
               "release-tag" / "spec-doc" / ...
    ID   string // path, repo URL, SHA, tenant UUID, ref, doc
           path
}
func (s Subject) String() string { return s.Kind + ":" + s.ID }

type Result struct {
    Decision Decision
    Evidence  string // URI or short message describing why
    DigestSHA [32]byte // hash of evaluator output, for
                       transition detection
}
```

`Subject.Kind + Subject.ID` MUST be **stable across runs** — together they form half of the (tenant, rule, subject) PK in `constitution_state`. The cherald rule catalogue (`cherald/internal/bindings/rules.go`) classifies subjects by Kind, including `spec-doc` (drives §11.4.73 drift) and the synthetic `revision-unchanged` Kind (a spec edited without a revision bump), plus `file`, `commit`, `gate`, `credential`, `pull-request`, `missing-revision-header`, `missing-companion-md`, `missing-catalogue-check`. `Result.DigestSHA` is opaque — the evaluator computes it however it likes (typically sha256 of the evidence + decision); the transition gate uses it to detect a `verdict-unchanged-but-rationale-changed` transition.

§2.3 The Evaluator interface (the primary extension point)

```
type Evaluator interface {
    RuleID()
        string //
        "§11.4.10"
    Severity()
        Severity //
        critical / high / middle / low
    PushTriggers()
        []string //
        CloudEvents `type` values that cause re-evaluation
    Subjects(ctx context.Context, tenantID uuid.UUID)
        ([]Subject, error) // what to evaluate
    Evaluate(ctx context.Context, s Subject, bundle BundleHash)
        (Result, error)
}
```

Lifetime contract (verbatim from source): `RuleID/Severity/PushTriggers` are STABLE for the process lifetime; `Subjects+Evaluate` may do I/O but MUST honor `ctx`; `Evaluate` MUST be concurrency-safe for the same evaluator; **panics inside Evaluate are caught** by the runner and translated to `DecisionError` without propagating to other evaluators.

§2.4 Registry — concurrent-safe `RuleID` → Evaluator map

```
func NewRegistry() *Registry
func (r *Registry) Register(e
    Evaluator) // panics on nil / empty
    RuleID / duplicate
func (r *Registry) Get(ruleID string) (Evaluator, bool)
func (r *Registry) Len() int
func (r *Registry) IterByMaxSeverity(max Severity)
    []Evaluator // snapshot, sorted by RuleID
func (r *Registry) All()
    []Evaluator // ==
    IterByMaxSeverity(SeverityCritical)
```

`Register` is **fail-fast**: a nil evaluator, an empty `RuleID()`, or re-registering the same `RuleID` with a *different* evaluator all panic — duplicate registration is a programming error, not a runtime condition. `IterByMaxSeverity` returns a **snapshot** sorted by `RuleID` (deterministic iteration), safe to range even under concurrent mutation; it is what the pull-trigger sweep uses to walk non-critical rules.

§3. BundleHash and the captureer

`bundle.go` defines the SHA-256 fingerprint of the constitution bundle at evaluation time — persisted on every event so a verdict is replayable against the exact rule text that produced it.

```

type BundleHash [sha256.Size]byte // 32 bytes
func (h BundleHash) Hex() string // lowercase hex
func (h BundleHash) String() string // == Hex
func (h BundleHash) IsZero() bool // "captureer hasn't run"
    probe

```

```

func Capture(path string) (BundleHash, error) //
    ErrBundleMissing if absent
func CaptureBytes(b []byte) BundleHash // in-memory
    variant

```

```

var ErrBundleMissing = errors.New("constitution bundle missing")

```

Capture returns `ErrBundleMissing` (wrapping) when the file is absent — callers translate that to `DecisionSkip`. Capture is intentionally **not** cached. For amortized repeated reads use the `Captureer`:

```

func NewCaptureer() *Captureer
func (c *Captureer) Hash(path string) (BundleHash, error) //
    mtime-keyed cache
func (c *Captureer) Invalidate(path
    string) // e.g. from a SIGHUP handler

```

The `Captureer` cache invalidates on **file mtime change** (re-hashing on a differing mtime) — so an editor save of `Constitution.md` resurfaces within one stat. `Invalidate` force-drops one path's entry. An all-zeros `IsZero()` is a safe uninitialized-state probe because a real all-zeros SHA-256 is cryptographically improbable.

§4. The `ConstitutionStore` transition gate

`state.go` defines the persisted state and the **three-axis transition** that is the heart of the §42.2 "transitions-only emission" discipline.

```

type Transition struct {
    OldDecision, NewDecision Decision
    OldDigest, NewDigest [32]byte
    OldBundleHash, NewBundleHash BundleHash
    Changed bool // true iff decision OR digest OR bundle-hash
        differs
    FirstSeen bool // true iff this (tenant, rule, subject) row
        didn't exist before
    At time.Time
}

```

```

type StateRow struct {
    TenantID uuid.UUID; RuleID, Subject string; Decision Decision
    Digest [32]byte; BundleHash BundleHash; EvidenceURI string;
    TransitionedAt time.Time
}

```

Changed == true iff **any** of the three axes moved: OldDecision != NewDecision, or OldDigest != NewDigest, or OldBundleHash != NewBundleHash. All three matter — a (rule, decision) verdict can stay the same while the rationale (DigestSHA) or the bundle revision changes, and that is still worth emitting. On the first sight of a (tenant, rule, subject) triple, FirstSeen=true and OldDecision is the zero value (DecisionPass) — so a brand-new *failing* row is a real transition; callers MUST check FirstSeen.

```
type ConstitutionStore interface {
    Record(ctx context.Context, tenantID uuid.UUID, ruleID,
        subject string,
        r Result, bundle BundleHash, evidenceURI string)
        (Transition, error)
    Get(ctx context.Context, tenantID uuid.UUID, ruleID, subject
        string) (StateRow, bool, error)
    List(ctx context.Context, tenantID uuid.UUID, q ListQuery)
        ([]StateRow, error)
}
```

Record is the only mutator — it UPSERTs the row keyed by (tenantID, ruleID, subject) and returns the observed Transition. List serves the /v1/compliance pull surface via ListQuery (RuleID, Subject, *Decision, Limit, plus the Wave-3a additions Since/Until — inclusive on both ends — and Offset, applied AFTER the deterministic ASC sort by TransitionedAt, so paginated callers walk oldest-first).

§5. The ModeLadder (allow / warn / enforce)

ladder.go defines the per-tenant per-rule enforcement-mode lookup that gates whether a transition merely records, also audits, or also emits to channels.

```
const (
    ModeAllow    Mode = iota // recorded only; no audit row; no
        channel emit
    ModeWarn          // recorded + audit row; no channel
        emit (pull-surface only)
    ModeEnforce       // recorded + audit row + channel
        emit (default for new bindings)
)
func (m Mode) String() string // allow / warn /
    enforce
func ParseMode(s string) (Mode, error) // inverse; ERRORS on
    unknown (config typos are loud)

type ModeLadder interface {
    Get(ctx context.Context, tenantID uuid.UUID, ruleID string)
        (Mode, error)
    Set(ctx context.Context, tenantID uuid.UUID, ruleID
        string, m Mode, by string) error
}
```

```
List(ctx context.Context, tenantID uuid.UUID)
    (map[string]Mode, error)
}
```

The **safe default** is load-bearing: `Get` returns `ModeEnforce` when no binding exists — a new rule enforces until an operator explicitly relaxes it. `Get` is on the **hot path of every evaluator call** (implementations **MUST** be sub-millisecond, lock-light); `Set` is rare (an admin REST action) and **MUST** be durable before returning, recording the operator identity by for the `constitution_bindings.mutated_by` audit column. `ParseMode` deliberately **ERRORS** rather than silently defaulting so a config typo is caught loudly.

§6. Event classes and the typed emit helpers

§6.1 The 13 classes

`doc.go` enumerates the closed set: **12 governance classes** (spec §42.2) plus **1 operational class** (HRD-090, operator decision 2026-05-29):

Constant	ce-type suffix	Class group
<code>ClassGateFailed</code>	<code>gate.failed</code>	governance
<code>ClassGateRecovered</code>	<code>gate.recovered</code>	governance
<code>ClassPolicyViolation</code>	<code>policy.violation</code>	governance
<code>ClassPolicyCleared</code>	<code>policy.cleared</code>	governance
<code>ClassHostSafetyBreach</code>	<code>host.safety.breach</code>	governance
<code>ClassRepoSafetyBreach</code>	<code>repo.safety.breach</code>	governance
<code>ClassCredentialLeak</code>	<code>credential.leak</code>	governance
<code>ClassBundleUpdated</code>	<code>bundle.updated</code>	governance
<code>ClassBundleUpdateFailed</code>	<code>bundle.update.failed</code>	governance
<code>ClassReleaseGateBlocked</code>	<code>release.gate.blocked</code>	governance
<code>ClassSpecRevisionDrift</code>	<code>spec.revision.drift</code>	governance
<code>ClassCatalogueMiss</code>	<code>catalogue.miss</code>	governance
<code>ClassQueueDeadLetter</code>	<code>queue.dead_letter</code>	operational (HRD-090)

`AllClasses() []string` returns all 13 in declaration order (12 governance + the 1 operational dead-letter) for boot-time validation, metrics-label-cardinality bounds, and "emits exactly these and no others" tests. `ClassQueueDeadLetter` is the one operational class: it fires when a background task is moved to the dead-letter table after exhausting retries or failing a §107 invariant — a queue-subsystem failure-terminal signal that lives alongside, not inside, the §42.2 governance enumeration.

§6.2 Per-class payload structs

`emit.go` defines a typed payload struct per class group: `GateEvent`, `PolicyEvent`, `SafetyEvent` (carries `BreachKind` — destructive-op/force-push/mem-budget/unauthorised-rm), `CredentialEvent` (carries `SourceOrigin`), `BundleEvent` (carries `OldBundle/NewBundle` + an `Error` for the failed variant), `ReleaseEvent` (`ReleaseRef+Reason`), `DriftEvent` (`SpecPath+OldRevision/NewRevision`), `CatalogueEvent` (`MissingRef`), and `QueueEvent` (`TaskID/FailureReason/FailureCount`).

§6.3 The EventEmitter interface — 13 typed methods

```
type EventEmitter interface {
    GateFailed(ctx, GateEvent) error
    GateRecovered(ctx, GateEvent) error
    PolicyViolation(ctx, PolicyEvent) error
    PolicyCleared(ctx, PolicyEvent) error
    HostSafetyBreach(ctx, SafetyEvent) error
    RepoSafetyBreach(ctx, SafetyEvent) error
    CredentialLeak(ctx, CredentialEvent) error
    BundleUpdated(ctx, BundleEvent) error
    BundleUpdateFailed(ctx, BundleEvent) error
    ReleaseGateBlocked(ctx, ReleaseEvent) error
    SpecRevisionDrift(ctx, DriftEvent) error
    CatalogueMiss(ctx, CatalogueEvent) error
    DeadLetter(ctx, QueueEvent) error
}
```

There are **13 emit methods** (one per class). Tests assert "emitter received exactly N events of class X" against this typed surface rather than inspecting raw bus events.

An optional capability lets the Runner capture the generated event ID for the audit trail:

```
type IDEmitter interface {
    PolicyViolationID(ctx, PolicyEvent) (uuid.UUID, error)
    PolicyClearedID(ctx, PolicyEvent) (uuid.UUID, error)
}
```

§6.4 NewEmitter

```
type EmitterConfig struct {
    Source string // "digital.vasic.herald/cherald" etc;
    REQUIRED
    Now func() time.Time // default time.Now().UTC()
    NewID func() string // default uuid.NewString
}
func NewEmitter(bus EventBus, cfg EmitterConfig) (EventEmitter,
    error)
```

`NewEmitter` returns the concrete `busEmitter` (the only `EventEmitter` implementation — it satisfies `IDEmitter`). It errors on a nil bus or an empty `Source`. Each emit composes the three-axis envelope (`rule_id`, `severity_category`, `decision_result`) plus `bundle_hash`, `traceparent`, `evidence_uri`, `tenant_id`, and (when `Transition.Changed`) `transition_from/transition_to`, JSON-marshals `{envelope, payload}` as `Event.Data`, sets the ce-type to `EventNamespace+"."+class`, and Publishes to the bus. Two source-level invariants worth knowing: `CredentialLeak` **always** emits at `SeverityCritical` regardless of the evaluator's severity; `CatalogueMiss` passes a zero `Transition{}` (it is a missing-reference signal, not a verdict transition).

§7. The EventBus shim (MemoryBus)

eventbus.go defines the in-process pub/sub whose shape mirrors digital.vasic.eventbus/pkg/event.Event so the M2 production swap is a rename + import-path change.

```
type Event struct {
    ID, Type, Source string; Time time.Time; Subject string
    Metadata map[string]string; Data []byte
}
type EventBus interface {
    Publish(ctx context.Context, ev Event) error
    Subscribe(t string) (*Subscription, error) // exact type, or
        "*" wildcard
    Close() error
    Metrics() BusMetrics
}
type BusMetrics struct {
    Published, Delivered, Dropped int64; Subscribers int
    PublishedByType map[string]int64
}
var ErrBusClosed = errors.New("constitution: event bus closed")

type MemoryBusConfig struct {
    PublishTimeout time.Duration // default 100ms
    BufferSize      int           // default 256
}
func NewMemoryBus(cfg MemoryBusConfig) *MemoryBus
```

MemoryBus is the shipped EventBus. Publish enqueues to every subscriber matching ev.Type plus every "*" subscriber, returning immediately after enqueue; on a subscriber whose buffer is full beyond PublishTimeout it **drops** (incrementing Dropped) rather than blocking. The per-type counter uses LoadOrStore to avoid a lost-increment race under concurrent publishers of a new type. Subscribe returns a Subscription whose .Cancel() is idempotent; Close() is idempotent and cancels every outstanding subscription, after which Publish/Subscribe both return ErrBusClosed. Metrics() is the anti-bluff hook — tests assert Published/Delivered actually moved, proving an emit really fired.

§8. The AuditStore write-through

audit.go defines the append-only audit trail behind RunOutcome.Audited. Prior to HRD-018 Audited was set true but **nothing was persisted** — a §107 PASS-bluff at the audit layer; HRD-018 made the write real.

```
type AuditRow struct {
    ID, TenantID uuid.UUID; RuleID, Subject string
    OldDecision *Decision // nil iff FirstSeen
    NewDecision Decision
    OldDigest *[32]byte // nil iff FirstSeen
    NewDigest [32]byte
```

```

    BundleHash BundleHash; EvidenceURI string
    EmittedEventID uuid.UUID // uuid.Nil for ModeWarn (audit-
        only)
    ModeAtEmission Mode; AuditedAt time.Time
}
type AuditStore interface {
    RecordAudit(ctx context.Context, row AuditRow) (uuid.UUID,
        error)
    ListAudit(ctx context.Context, tenantID uuid.UUID, q
        AuditQuery) ([]AuditRow, error)
}

```

One `AuditRow` is written for every **CHANGED** transition whose mode is `ModeWarn` or `ModeEnforce` (`ModeAllow` is recorded in state only — no audit, no emit). `EmittedEventID` is the EXACT event ID that fanned out for `ModeEnforce` rows (load-bearing for replay correlation, §42.1.3) and `uuid.Nil` for `ModeWarn` rows (audit-only, never pushed). `RecordAudit` is the only mutator and is **append-only** — there is no `Update/Delete` (the Postgres backend's RLS policy forbids `UPDATE/DELETE` at the row level). `ListAudit` serves `/v1/compliance/audit` (`AuditQuery` filters `RuleID/Subject/Since/Until/Limit/Offset`, sorted `AuditedAt DESC` — newest-first).

§9. The Runner orchestrator

`runner.go` composes the `Registry`, `ModelLadder`, `ConstitutionStore`, `EventEmitter`, and `AuditStore` into the canonical flow.

```

func NewRunner(reg *Registry, ladder ModelLadder, store
    ConstitutionStore,
    emitter EventEmitter, audit AuditStore) (*Runner,
    error)
func (r *Runner) Run(ctx context.Context, e Evaluator, tenantID
    uuid.UUID,
    subject Subject, bundle BundleHash)
    (RunOutcome, error)

type RunOutcome struct {
    Evaluator string; Subject Subject; Decision Decision; Mode
    Mode
    Transition Transition; Emitted, Audited bool; PanicValue
    string
}

```

All five dependencies are required (a nil `AuditStore` is a hard error — HRD-018). The canonical Run flow:

1. **evaluate** inside a panic-recovery wrapper (`safeEvaluate`): a panicking evaluator yields `DecisionError + PanicValue` and **does not propagate** to other evaluators; the error state is still `Recorded` so a future run sees the regression. An error-returning (non-panicking) evaluator also becomes `DecisionError`.

2. **record** via `store.Record`, computing the `Transition`. State is persisted even for transitions that won't emit (`mode=allow`, or `!Changed`) — so "we always know what state every (`tenant`, `rule`, `subject`) is in" holds (§42.1.2).
3. **transitions-only gate**: if `!trans.Changed`, stop here (no audit, no emit).
4. **gate by mode**: `ModeAllow` → stop; `ModeWarn` → write an audit row with `EmittedEventID = uuid.Nil`, no emit; `ModeEnforce` → emit then audit. For enforce, the emit picks `PolicyCleared` for a `DecisionPass/DecisionWarn` transition and `PolicyViolation` otherwise; the ID-returning `IDEmitter` variant is used when available so the audit row records the exact emitted event ID, and the audit row is written **after** a successful emit.

Run returns errors loud (`ladder.Get`, `store.Record`, `emit` all propagate after state is recorded) — defence-in-depth, never a silent pass.

§10. The CloudEvents v1.0 adapter

`cloudevents.go` bridges the in-process `Event` to a CNCF CloudEvents v1.0 envelope for HTTP egress, using github.com/cloudevents/sdk-go/v2.

```
func ToCloudEvent(ev Event) (cloudevents.Event, error)
func FromCloudEvent(ce cloudevents.Event) (Event, error)
var ErrUnsupportedSpec = errors.New("unsupported CloudEvents SpecVersion")
```

`ToCloudEvent` sets `SpecVersion` (`VersionV1`), copies `ID/Type/Source/Time/Subject`, sets the JSON data (`cloudevents.ApplicationJSON`), maps every `Metadata` entry to a CE **extension** (keys normalized via `normalizeExtKey` — strip underscores/hyphens/dots, lowercase, per CE v1.0 §3 attribute-name rules, e.g. `rule_id` → `ruleid`), and **validates** before returning. `FromCloudEvent` rejects any non-v1.0 spec version with `ErrUnsupportedSpec`, validates, and copies extensions back into `Metadata` (stringifying non-string values).

§11. How cherald consumes the module

cherald is the **compliance flavor**. It wires `commons_constitution` into three live REST surfaces, declared in `cherald/internal/http/routes.go`:

Method + Path	cherald handler	Backed by
GET /v1/compliance	<code>compliance.Handler(store)</code>	<code>ConstitutionStore.List</code>
GET /v1/compliance/modes	<code>modes.ListHandler(ladder)</code>	<code>ModeLadder.List</code>
GET /v1/compliance/modes/:rule_id	<code>modes.GetHandler(ladder)</code>	<code>ModeLadder.Get</code>
PUT /v1/compliance/modes/:rule_id	<code>modes.PutHandler(ladder)</code>	<code>ModeLadder.Set</code>
	<code>compliance.EvaluateHandler(pipeline)</code>	the bindings <code>Pipeline</code> (write side)

Method + Path	cherald handler	Backed by
POST /v1/ compliance/ evaluate		

§11.1 The read surface — GET /v1/compliance

`compliance.Handler` (`cherald/internal/compliance/handler.go`) extracts the tenant from `commons_auth` JWT claims, maps the query params (`rule_id`, `subject`, `decision` with `allow→pass/deny→fail` aliasing, `since/until` RFC3339, `page/page_size` with `defaultPageSize=50`, `maxPageSize=200`) onto a `constitution.ListQuery`, and calls `store.List`. Response rows surface `rule_id/subject/decision/digest_sha/bundle_hash/evidence_uri/transitioned_at` plus a pre-pagination total. Anti-bluff posture: 401 on missing/bad auth, 400 on a malformed param, 500 on a store error — never a success on bad input.

§11.2 The mode-ladder admin surface — /v1/compliance/modes

`modes.PutHandler` (`cherald/internal/modes/handler.go`) lets an operator flip a rule's mode (`allow/warn/enforce`) **per tenant without a redeploy** — the new mode takes effect on the NEXT evaluation because the ladder reads `constitution_bindings` at evaluation time. It validates the body via `constitution.ParseMode`, records the operator identity from the JWT sub claim (`mutated_by`), and calls `ladder.Set`. `ListHandler` returns every binding plus the `unbound_default = ModeEnforce.String()` note; `GetHandler` returns one rule's effective mode (`enforce` when unbound). Posture: 401 on bad auth, 400 on a malformed `rule_id/body`, 502 on a store error.

§11.3 The write side — the bindings Pipeline + audit write-through

`cherald/internal/bindings/bindings.go` defines the cherald rule catalogue and a cherald-local Pipeline that reuses the Runner's exact gate semantics but routes the emit through each rule's declared `EventClass` (the Runner hardcodes the policy class; cherald binds rules to five classes — `policy.violation`, `gate.failed/gate.recovered`, `credential.leak`, `spec.revision.drift`, `catalogue.miss`). `RuleSpec` carries the `RuleID`, `Severity`, `DefaultMode`, `EventClass`, the `CheckFunc`, and `SubjectKinds`; `Binding` adapts a `RuleSpec` to the `Evaluator` interface; `NewPipeline` registers every `CheraldRules()` entry into a fresh `Registry`. `Pipeline.EvaluateSubject` runs the full `evaluate→record→gate→emit→audit` flow against the REAL emitter + REAL store + REAL `AuditStore` — the persisted side-effects ARE the positive runtime evidence (§107 anti-bluff). The audit write-through is the same `AuditRow` shape as `Runner.writeAudit`: written for every CHANGED `warn/enforce` transition, with the EXACT `EmittedEventID` captured via `IDemitter` for the policy class.

§12. Backends summary (Memory vs Postgres)

Each persistence interface ships an in-memory test backend (M1) and an RLS-guarded Postgres backend (M2/M3):

Interface	Memory (M1)	Postgres (M2/M3)
ModeLadder	<code>ladder.NewMemory()</code> (<code>ladder/memory.go</code>) —	<code>ladder.NewPostgres(db)</code> (<code>ladder/postgres.go</code>) —

Interface	Memory (M1)	Postgres (M2/M3)
ConstitutionStore	RWMutex map, records a Mutation audit slice state.NewMemory() (state/memory.go) — sync map keyed (tenant, rule, subject)	constitution_bindings, per-call tx state.NewPostgres(db) (state/postgres.go) — RLS UPSERT into constitution_state
AuditStore	state.NewMemoryAudit() (state/audit_memory.go) — append-only slice	state.NewPostgresAudit(db) (state/audit_postgres.go) — INSERT-only, RLS forbids UPDATE/DELETE
EventBus	NewMemoryBus(cfg)	swap for Helix digital.vasic.eventbus (interface is a subset)

The Postgres backends take a `digital.vasic.database/pkg/database.Database` (caller owns connection lifecycle) and run each call inside its own tenant-scoped transaction. The Redis read-cache `ModelLadder` wrapper (M3, 60s TTL — `ladder/redis_cache.go`) is referenced in the source doc comment but is future work.

§13. Testing notes

Tests run with no external services (Memory backends + temp files); the Postgres-backed tests are gated on a live container:

```
go test -race -count=1 ./commons_constitution/...
```

Test file	Proves
evaluator_test.go	Registry register/get/iter + Severity/Decision string round-trips
bundle_test.go	Capture/CaptureBytes/Captureerr mtime-cache + ErrBundleMissing
emit_test.go + queue_emit_test.go	each typed emit composes the right ce-type + envelope; DeadLetter
eventbus_test.go	publish/subscribe/wildcard, overflow-drop, Metrics(), Close → ErrBusClosed
cloudevents_test.go	ToCloudEvent/FromCloudEvent round-trip + ErrUnsupportedSpec
audit_test.go	AuditStore append-only + ListAudit ordering/filtering
integration_test.go + postgres_integration_test.go	full Runner flow on Memory and on a real RLS Postgres
audit_stress_chaos_test.go	§11.4.85 stress + chaos at the audit layer

Anti-bluff observations worth preserving: `MemoryBus.Metrics()` is used as the positive-evidence anchor (an emit test asserts `Published/Delivered` moved, not just "no error"); the Postgres tests exercise the real RLS append-only policy (a DELETE attempt must fail), not a mock; the panic-isolation tests assert a panicking evaluator yields `DecisionError` + a recorded error state without taking down its neighbours.

§14. References

- Source: `commons_constitution/*.go` — `doc.go`, `evaluator.go`, `bundle.go`, `state.go`, `ladder.go`, `emit.go`, `eventbus.go`, `audit.go`, `runner.go`, `cloudevents.go`, plus `ladder/ + state/ backends`.
- Spec: `docs/specs/mvp/specification.V4.md` (predecessor §42 Constitution-flavor binding catalogue + §44 Foundation implementation contract; the in-code references cite the V3 lineage §42.1–§42.5 / §44).
- Catalogue-Check: `docs/catalogue-checks/HRD-018-foundation.md` (the no-match decision that made the Evaluator framework + BundleHash captureer + ModeLadder bespoke, and the audit write-through that HRD-018 closed).
- Herald constitution: `docs/guides/HERALD_CONSTITUTION.md` §107 (the anti-bluff covenant every PASS in this module is held to).
- Consumer: `cherald/internal/{http,compliance,modes,bindings}/` — the `/v1/compliance`, `/v1/compliance/modes`, and `/v1/compliance/evaluate` surfaces.
- Dependencies: `github.com/cloudevents/sdk-go/v2 v2.15.2`, `github.com/google/uuid v1.6.0`, `digital.vasic.database` (`commons_constitution/go.mod`).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited source files as of 2026-05-31.

Verified 2026-05-31: internal doc — no external online sources. Behavioural claims derive from `commons_constitution/*.go` (incl. `ladder/ + state/ backends`) and the cherald consumers under `cherald/internal/` (read 2026-05-31). The only third-party dependencies are `github.com/cloudevents/sdk-go/v2 v2.15.2` and `github.com/google/uuid v1.6.0`, both version-pinned in `commons_constitution/go.mod` — the CloudEvents v1.0 spec-version + extension-name rules cited in §10 are the ones the vendored, pinned SDK enforces. Re-verify on a `cloudevents/sdk-go` major-version bump or a `commons_constitution` API change.